

45-internationalization

□ Internasionalisasi (i18n) — Multi-Language React Application



Figure 1: Pexels Banner

Level: Mahir

Total Jam: 20 Jam (4 sesi × 5 jam)

Prasyarat: Node.js, React/framework modern (Modul 20), TypeScript dasar

Tipe Modul: Baru — Internationalization khusus frontend + backend

□ Materi

Sesi	Topik	Durasi
01	i18n Concepts — locale, translation file, ICU message, pluralization	5 jam
02	React i18n — react-i18next, useTranslation, Trans, namespace, SSR	5 jam
03	RTL & Backend — bidirectional layout, Accept-Language, DB multilingual	5 jam
04	Production i18n — SaaS TMS, i18n linting, code splitting, SEO hreflang	5 jam

□ Output Akhir

Aplikasi React multi-language dengan dukungan:

- **English (en-US)** — default
- **Bahasa Indonesia (id-ID)** — LTR locale
- **Arabic (ar-SA)** — RTL locale, full bidirectional styling

Fitur: - Runtime language switching tanpa reload - RTL layout otomatis (CSS logical properties) - Pluralization & interpolation (ICU Message-Format) - Lazy-loaded translation per locale (code splitting) - i18n middleware backend Express - Database multilingual (PostgreSQL JSONB) - SEO hreflang tag - Translation workflow (Dev → Translation → Review → Deploy)

□ AI Prompt Exercises

Setiap sesi menyertakan 4 latihan berbasis AI prompt. Mahasiswa mengerjakan langsung dengan bantuan AI (ChatGPT, Claude, GitHub Copilot) sebagai pair programmer.

Contoh prompt per sesi:

1. "Bantu saya setup folder locales dengan struktur JSON untuk 3 bahasa: en, id, ar. Sertakan namespace common, home, auth dengan 5 key masing-masing."

2. "Tulis komponen LanguageSwitcher React dengan react-i18next yang menampilkan flag emoji + nama bahasa. Gunakan use-Translation hook."
3. "Buat middleware Express yang mendeteksi Accept-Language header dan menyediakan locale di req object."
4. "Implementasi lazy loading translations dengan React.lazy + Suspense per locale. Beri fallback loading spinner."

□ Struktur Proyek Akhir

```
project/
├── public/
│   └── locales/
│       ├── en/
│       │   ├── common.json
│       │   ├── home.json
│       │   └── auth.json
│       ├── id/
│       │   ├── common.json
│       │   ├── home.json
│       │   └── auth.json
│       └── ar/
│           ├── common.json
│           ├── home.json
│           └── auth.json
├── src/
│   ├── i18n/
│   │   ├── config.ts
│   │   ├── i18n.d.ts
│   │   └── locale-detector.ts
│   ├── components/
│   │   ├── LanguageSwitcher.tsx
│   │   └── RTLWrapper.tsx
│   ├── pages/
│   ├── backend/
│   │   ├── middleware/i18n.ts
│   │   └── models/locale.ts
│   └── i18next-scanner.config.js
```

Sesi 01: Konsep Dasar Internasionalisasi (i18n)

Durasi: 5 Jam

□ Tujuan Pembelajaran

Setelah sesi ini mahasiswa mampu:

- Membedakan i18n dan l10n
 - Memahami format locale (en-US, id-ID, ar-SA)
 - Menyusun struktur file translasi JSON (nested keys)
 - Mengimplementasikan interpolation & pluralization
 - Menggunakan ICU Message Format untuk translasi kompleks
-

□ Materi

1. i18n vs l10n

Istilah	Arti	Contoh
i18n (Internationalization)	Proses desain aplikasi agar siap diadaptasi ke berbagai bahasa/region	Memisahkan string dari kode, pakai locale IDs
l10n (Localization)	Proses adaptasi konten ke bahasa/lokal spesifik	Menerjemahkan string ke Bahasa Indonesia

i18n = kerangka teknis. l10n = konten terjemahan.

Best practice: Tulis kode dg i18n sejak awal. Migrasi aplikasi existing ke i18n jauh lebih mahal.

2. Format Locale

Standar: **BCP 47** — language-REGION

Locale	Bahasa	Region	Catatan
en-US	English	United States	default
en-GB	English	United Kingdom	colour vs color
id-ID	Bahasa Indonesia	Indonesia	LTR
ar-SA	Arabic	Saudi Arabia	RTL
zh-CN	Chinese (Simplified)	China	
ja-JP	Japanese	Japan	

```
// locale constant
export const SUPPORTED_LOCALES = ['en-US', 'id-ID', 'ar-SA'] as const;
export type Locale = (typeof SUPPORTED_LOCALES)[number];

export const DEFAULT_LOCALE: Locale = 'en-US';

export const localeConfig: Record<Locale, {
  label: string;
  dir: 'ltr' | 'rtl';
  flag: string;
}> = {
  'en-US': { label: 'English', dir: 'ltr', flag: '🇺🇸' },
  'id-ID': { label: 'Bahasa Indonesia', dir: 'ltr', flag: '🇮🇩' },
  'ar-SA': { label: 'العربية', dir: 'rtl', flag: '🇸🇦' },
};
```

3. Struktur File Translasi

Rekomendasi: **JSON flat dengan namespaces.**

```
public/locales/
├── en/
│   ├── common.json    # tombol, label, error umum
│   ├── home.json      # halaman home
│   └── auth.json       # login, register, logout
├── id/
│   ├── common.json
│   ├── home.json
│   └── auth.json
└── ar/
    ├── common.json
    ├── home.json
    └── auth.json
```

Format JSON — nested keys:

```
{
  "greeting": "Hello",
  "nav": {
    "home": "Home",
    "about": "About Us",
    "contact": "Contact"
  },
  "error": {
    "notFound": "Page not found",
    "serverError": "Something went wrong. Please try again later."
  }
}
```

```
}
```

Akses via dot notation: `t('greeting'), t('nav.home'), t('error.notFound')`

4. Interpolation & Pluralization

Interpolation

```
{
  "welcomeUser": "Welcome, {{name}}!",
  "itemsInCart": "You have {{count}} items in your cart."
}

t('welcomeUser', { name: 'Alice' });
// → "Welcome, Alice!"
t('itemsInCart', { count: 3 });
// → "You have 3 items in your cart."
```

Pluralization — ICU MessageFormat

```
{
  "cows": "{count, plural, one {# cow} other {# cows}}",
  "apples": "{count, plural, =0 {No apples} one {# apple} other {# apples}}"
}

t('cows', { count: 1 }); // → "1 cow"
t('cows', { count: 5 }); // → "5 cows"
t('apples', { count: 0 }); // → "No apples"
```

Plural rule per locale: ICU otomatis deteksi cardinal plural rule CLDR. Bahasa Indonesia pakai other aja — nggak punya one/few/many.

```
{
  "buku": "{count, plural, other {# buku}}"
}
```

5. ICU Message Format — Fitur Lengkap

Select (gender, pronoun)

```
{
  "partyInvitation": "{name} {gender, select, male {invited you to his party} fe"
}
```

Nested plural + select

```
{
  "notification": "You have {count, plural, =0 {no notifications} one {# notificat"
}
```

Number formatting

```
{
  "price": "The price is {price, number, ::currency/USD}"
}
```

ICU memungkinkan kombinasi kompleks — hindari nested berlebihan di kode, pecah jadi beberapa key.

□ Kode Praktik

Setup Folder Proyek

```
mkdir -p public/locales/{en,id,ar}
```

Contoh File: public/locales/en/common.json

```
{
  "app": {
    "name": "MyApp",
    "tagline": "Built for everyone"
  },
  "nav": {
    "home": "Home",
    "about": "About",
    "language": "Language"
  },
  "greeting": "Hello, {{name}}!",
  "items": "{count, plural, =0 {No items} one {# item} other {# items}}",
  "notification": {
    "title": "Notifications",
    "new": "You have {count, plural, =0 {no new notifications} one {# new notifi"
  }
}
```

Contoh File: public/locales/id/common.json

```
{
  "app": {
    "name": "MyApp",
```

```

    "tagline": "Dibangun untuk semua"
  },
  "nav": {
    "home": "Beranda",
    "about": "Tentang",
    "language": "Bahasa"
  },
  "greeting": "Halo, {{name}}!",
  "items": "{count, plural, other {# item}}",
  "notification": {
    "title": "Notifikasi",
    "new": "Anda memiliki {count, plural, other {# notifikasi baru}}"
  }
}

```

Deteksi Locale Browser

```

// src/i18n/locale-detector.ts
import { SUPPORTED_LOCALES, DEFAULT_LOCALE, type Locale } from './config';

export function detectBrowserLocale(): Locale {
  const locales = navigator.languages ?? [navigator.language];

  for (const lang of locales) {
    // exact match: en-US, id-ID
    if (SUPPORTED_LOCALES.includes(lang as Locale)) {
      return lang as Locale;
    }
    // partial match: en → en-US
    const base = lang.split('-')[0];
    const match = SUPPORTED_LOCALES.find((l) => l.startsWith(base));
    if (match) return match;
  }

  return DEFAULT_LOCALE;
}

```

Format ICU di JavaScript Native (opsional)

```

// tanpa library – Intl.PluralRules
const pr = new Intl.PluralRules('id-ID');
console.log(pr.select(0)); // "other"
console.log(pr.select(1)); // "other"
console.log(pr.select(2)); // "other"

```

```
const prEn = new Intl.PluralRules('en-US');
console.log(prEn.select(0)); // "other"
console.log(prEn.select(1)); // "one"
console.log(prEn.select(2)); // "other"
```

□ AI Prompt Exercises

Latihan 1: Struktur Locale Folder

Prompt untuk AI:

“Saya perlu setup folder locales untuk aplikasi React multi-bahasa. Buatlah struktur folder public/locales/{en,id,ar} dengan namespace: common, home, auth. Masing-masing minimal 10 key. Gunakan nested JSON. Sertakan interpolation ({ {name} }, { {count} }) dan pluralization ICU.”

Kriteria: Semua file valid JSON, konsisten antar bahasa, mencakup greeting, nav, error, button, form label.

Latihan 2: Format ICU Kompleks

Prompt untuk AI:

“Bantu saya membuat key translasi ICU untuk skenario: notifikasi undangan pesta. Harus kombinasi plural (jumlah tamu), gender (host gender), dan waktu relatif (hari ini, kemarin). Gunakan ICU MessageFormat select dan plural.”

Kriteria: Satu key tunggal, minimal 4 varian output berbeda tergantung parameter.

Latihan 3: Locale Detector & Validator

Prompt untuk AI:

“Tulis fungsi TypeScript yang menerima Accept-Language header string (contoh: 'id-ID,id;q=0.9,en-US;q=0.8,en;q=0.7') dan mengembalikan locale terbaik dari daftar ['en-US', 'id-ID', 'ar-SA']. Implementasikan quality value (q) parsing dan fallback ke default.”

Kriteria: Handle edge case (no match, malformed q value), gunakan generic type.

Latihan 4: Number & Date Formatter

Prompt untuk AI:

“Buat utility function TypeScript untuk memformat number (mata uang) dan date (long, short, relative) berdasarkan locale. Gunakan Intl.NumberFormat dan Intl.DateTimeFormat native. Dukung locale: en-US, id-ID, ar-SA.”

Kriteria: Output sesuai locale (ID: Rp 1.000, EN: \$1,000, AR: ر.س. ١٠٠٠). Tanggal pakai format lokal.

□ Evaluasi

Indikator	Deskripsi
Struktur file JSON	Hierarki benar, namespace terpisah, valid JSON
ICU format	Select, plural, interpolation berfungsi
Locale detection	Prioritas navigator.languages, fallback default
Code splitting siap	Namespace per domain (common, home, auth)

Sesi 02: React i18n dengan react-i18next

Durasi: 5 Jam

□ Tujuan Pembelajaran

Setelah sesi ini mahasiswa mampu:

- Mengintegrasikan react-i18next ke aplikasi React
- Menggunakan useTranslation hook di komponen
- Merender HTML di translasi dengan Trans component
- Membuat language switcher interaktif
- Mengelola multiple namespaces (common, home, auth)
- Mengimplementasikan SSR dengan i18n (Next.js)

□ Materi

1. Setup react-i18next

```
npm install react-i18next i18next i18next-browser-languagedetector i18next-http-
```

Konfigurasi i18n:

```
// src/i18n/config.ts
import i18n from 'i18next';
import { initReactI18next } from 'react-i18next';
import Backend from 'i18next-http-backend';
import LanguageDetector from 'i18next-browser-languagedetector';

export const SUPPORTED_LOCALES = ['en', 'id', 'ar'] as const;
export type Locale = (typeof SUPPORTED_LOCALES)[number];
export const DEFAULT_LOCALE: Locale = 'en';

i18n
  .use(Backend) // load JSON via HTTP
  .use(LanguageDetector) // auto-detect browser locale
  .use(initReactI18next) // bind ke React
  .init({
    fallbackLng: DEFAULT_LOCALE,
    debug: process.env.NODE_ENV === 'development',

    interpolation: {
      escapeValue: false, // React already escapes XSS
    },

    // namespace default
    defaultNS: 'common',

    // backend config
    backend: {
      loadPath: '/locales/{{lng}}/{{ns}}.json',
    },

    // language detector config
    detection: {
      order: ['querystring', 'localStorage', 'navigator', 'cookie', 'htmlTag'],
      caches: ['localStorage'],
    },
  });

export default i18n;
```

Initialisasi di entry point:

```
// src/main.tsx - Vite
import './i18n/config'; // import i18n sebelum React

ReactDOM.createRoot(document.getElementById('root')).render(
```

```

    <React.StrictMode>
      <App />
    </React.StrictMode>,
  );

```

Catatan: Import i18n config SEBELUM komponen Root — pastikan translation siap saat render pertama.

2. useTranslation Hook

```

import { useTranslation } from 'react-i18next';

function Welcome() {
  const { t, i18n } = useTranslation('common');
  //           ^^ instance i18n - ganti bahasa, dll

  const currentLang = i18n.language; // 'en' | 'id' | 'ar'
  const dir = i18n.dir();           // 'ltr' | 'rtl'

  return (
    <div>
      <h1>{t('greeting', { name: 'Alice' })}</h1>
      <p>{t('items', { count: 5 })}</p>
      <p>Direction: {dir}</p>
    </div>
  );
}

```

Multiple namespaces:

```

const { t } = useTranslation(['common', 'home', 'auth']);

t('common:greeting'); // prefix namespace
t('home:title');
t('auth:login.btn');

```

Default namespace tidak perlu prefix:

```

const { t } = useTranslation(['common', 'home']);
// common adalah defaultNS → t('greeting') cukup
// home harus explicit → t('home:title')

```

3. Trans Component — HTML di Translations

Gunakan Trans saat value translasi mengandung HTML atau React elements.

```

{
  "readMore": "Read <link>more about i18n</link> on our blog.",
  "terms": "I agree to the <1>Terms of Service</1> and <2>Privacy Policy</2>."
}

import { Trans } from 'react-i18next';

function BlogPost() {
  return (
    <p>
      <Trans i18nKey="readMore">
        Read <a href="/blog">more about i18n</a> on our blog.
      </Trans>
    </p>
  );
}

```

Dengan components:

```

function TermsCheckbox() {
  return (
    <label>
      <Trans
        i18nKey="terms"
        components={[
          <a key="1" href="/terms" />,
          <a key="2" href="/privacy" />,
        ]}
      />
    </label>
  );
}

```

Tag <1>, <2> di JSON — urut sesuai array components.

4. Language Switcher

```

// src/components/LanguageSwitcher.tsx
import { useTranslation } from 'react-i18next';

const languages = [
  { code: 'en', label: 'English', flag: '🇺🇸' },
  { code: 'id', label: 'Indonesia', flag: '🇮🇩' },
  { code: 'ar', label: 'العربية', flag: '🇸🇦' },
] as const;

export function LanguageSwitcher() {

```

```

const { i18n } = useTranslation();

const changeLanguage = (lng: string) => {
  i18n.changeLanguage(lng);
  // optional: set html lang & dir
  document.documentElement.lang = lng;
  document.documentElement.dir = i18n.dir();
};

return (
  <div className="language-switcher">
    {languages.map(({ code, label, flag }) => (
      <button
        key={code}
        onClick={() => changeLanguage(code)}
        disabled={i18n.language === code}
        className={
          i18n.language === code ? 'active' : ''
        }
      >
        {flag} {label}
      </button>
    ))}
  </div>
);
}

```

CSS:

```

.language-switcher {
  display: flex;
  gap: 0.5rem;
}
.language-switcher button.active {
  font-weight: bold;
  border-bottom: 2px solid currentColor;
}

```

5. Namespace Strategy

Namespace	Isi	Contoh Key
common	Global: nav, footer, error, button	nav.home, error.general

Namespace	Isi	Contoh Key
home	Halaman utama	hero.title, features.list
auth	Login, register, reset password	login.btn, register.emailLabel
dashboard	(jika ada)	stats.title, chart.label

Alasan namespace: (1) Code splitting — load per page, (2) Translator dapat fokus per domain, (3) Conflict key berkurang.

6. SSR dengan i18n (Next.js App Router)

```
// app/[locale]/layout.tsx – Next.js App Router
import { NextIntlClientProvider } from 'next-intl';
import { notFound } from 'next/navigation';

const locales = ['en', 'id', 'ar'];

export default async function LocaleLayout({
  children,
  params: { locale },
}: {
  children: React.ReactNode;
  params: { locale: string };
}) {
  if (!locales.includes(locale)) notFound();

  let messages;
  try {
    messages = (await import(`../../messages/${locale}.json`)).default;
  } catch {
    notFound();
  }

  return (
    <html lang={locale} dir={locale === 'ar' ? 'rtl' : 'ltr'}>
      <body>
        <NextIntlClientProvider locale={locale} messages={messages}>
          {children}
        </NextIntlClientProvider>
      </body>
    </html>
  );
}
```

```

    </html>
  );
}

```

Static Generation:

```

// generateStaticParams
export function generateStaticParams() {
  return locales.map((locale) => ({ locale }));
}

```

□ Kode Praktik

Setup Vite + react-il8next (langkah lengkap)

```

npm create vite@latest il8n-app -- --template react-ts
cd il8n-app
npm install react-il8next il8next il8next-browser-languagedetector il8next-http-

```

Struktur file setelah setup

```

src/
├── il8n/
│   └── config.ts
├── components/
│   ├── LanguageSwitcher.tsx
│   └── Greeting.tsx
├── App.tsx
└── main.tsx
public/
└── locales/
    ├── en/common.json
    ├── en/home.json
    ├── id/common.json
    ├── id/home.json
    ├── ar/common.json
    └── ar/home.json

```

Contoh: src/components/Greeting.tsx

```

import { useState } from 'react';
import { useTranslation, Trans } from 'react-il8next';

export function Greeting() {
  const { t } = useTranslation('common');

```

```

const [count, setCount] = useState(0);

return (
  <div>
    <h1>{t('greeting', { name: 'World' })}</h1>

    <p>{t('items', { count })}</p>
    <button onClick={() => setCount((c) => c + 1)}>+</button>
    <button onClick={() => setCount((c) => Math.max(0, c - 1))}>-</button>

    <p>
      <Trans i18nKey="learnMore">
        Learn more at <a href="https://react.i18next.com">react.i18next.com</a>
      </Trans>
    </p>
  </div>
);
}

```

□ AI Prompt Exercises

Latihan 1: Setup i18n Config

Prompt untuk AI:

“Bantu saya setup react-i18next di aplikasi React Vite TypeScript. Konfigurasi: fallback ke ‘en’, support en/id/ar, deteksi browser via localStorage, load path ‘/locales/{lng}/{ns}.json’, namespace default ‘common’. Tulis file i18n/config.ts lengkap.”

Kriteria: Import urutan benar, debug mode hanya di development, escapeValue: false.

Latihan 2: Komponen Form Multi-Bahasa

Prompt untuk AI:

“Buat komponen React LoginForm dengan react-i18next. Input: email, password. Tombol: login. Semua label dan placeholder dari translasi namespace ‘auth’. Tampilkan error validasi juga dari translasi. Gunakan useTranslation hook.”

Kriteria: Label, placeholder, error message, tombol — semua dari JSON. Fallback jika key tidak ditemukan.

Latihan 3: Trans Component untuk Rich Text

Prompt untuk AI:

“Buat komponen Footer yang menggunakan Trans component. Di dalamnya ada link ke About, Privacy Policy, Terms of Service — semuanya dari string translasi. Gunakan component mapping dengan tag <1>, <2>, <3>.”

Kriteria: Tiga link berbeda di satu kalimat translasi. Link terbuka di tab baru. Styling inline atau Tailwind.

Latihan 4: Namespace Loading + Skeleton

Prompt untuk AI:

“Saya punya namespace ‘dashboard’ yang besar — tidak mau di-load di awal. Buat pola lazy loading namespace: ketika komponen Dashboard mount, panggil `i18n.loadNamespaces(['dashboard'])`. Tampilkan skeleton loading selama namespace belum siap.”

Kriteria: Gunakan state, loading skeleton komponen terpisah, error boundary jika gagal load.

□ Evaluasi

Indikator	Deskripsi
Setup i18n	Backend, detector, React binding benar
useTranslation	Interpolation, pluralization, namespace berfungsi
Trans component	HTML/component di translasi render benar
Language switcher	Ganti bahasa tanpa reload, update html lang/dir
Lazy namespace	Code splitting per namespace, loading state

Sesi 03: RTL Layout & Backend i18n

Durasi: 5 Jam

□ Tujuan Pembelajaran

Setelah sesi ini mahasiswa mampu:

- Mengimplementasikan RTL layout dengan `dir="rtl"`

- Menggunakan CSS logical properties untuk bidirectional styling
- Menangani pertimbangan tipografi Arab/Hebrew
- Membangun API translation dengan header Accept-Language
- Membuat i18n middleware Express
- Mendesain database schema untuk konten multilingual

□ Materi

1. RTL Layout — dir="rtl"

HTML global attribute dir:

```
<html lang="ar" dir="rtl">
```

Efek: - Teks rata kanan secara default - Arah kolom tabel terbalik - Scrollbar di kiri (browser-dependent) - Urutan kolom flex/grid berdasarkan direction

React effect:

```
// Trigger saat ganti bahasa
const changeLanguage = (lng: string) => {
  i18n.changeLanguage(lng);
  document.documentElement.lang = lng;
  document.documentElement.dir = i18n.dir();
};
```

Auto-direction via CSS logical properties:

Physical	Logical	Efek
left	inline-start	Kiri di LTR, kanan di RTL
right	inline-end	Kanan di LTR, kiri di RTL
margin-left	margin-inline-start	Margin arah awal
padding-right	padding-inline-end	Padding arah akhir
border-left	border-inline-start	Border sisi awal
text-align: left	text-align: start	Rata awal paragraf
text-align: right	text-align: end	Rata akhir paragraf

Contoh: dari physical → logical

```
/* □ Physical – rusak di RTL */
.card {
  margin-left: 1rem;
  padding-right: 0.5rem;
  text-align: left;
```

```

    border-left: 3px solid blue;
}

/* □ Logical – works both directions */
.card {
    margin-inline-start: 1rem;
    padding-inline-end: 0.5rem;
    text-align: start;
    border-inline-start: 3px solid blue;
}

```

Flex/Grid auto-adjust:

```

.nav {
    display: flex;
    gap: 1rem;
    /* Urutan item otomatis terbalik di RTL */
    /* tidak perlu mengubah apapun */
}

/* Kecuali urutan visual harus tetap */
.nav .logo {
    order: 1; /* force order regardless of dir */
}

```

2. Bidirectional Styling — Gambar & Icon

Mirror icon untuk RTL:

```

/* Icon yang punya arah (arrow, chevron) */
.icon-arrow {
    /* default: pointer left */
}

[dir="rtl"] .icon-arrow {
    transform: scaleX(-1); /* mirror horizontal */
}

```

Background positioning:

```

.hero {
    background: url('/bg.jpg') no-repeat;
    background-position: left center;
}

[dir="rtl"] .hero {
    background-position: right center;
}

```

CSS custom property approach (cleaner):

```
:root {
  --bg-pos: left center;
}

[dir="rtl"] {
  --bg-pos: right center;
}

.hero {
  background-position: var(--bg-pos);
}
```

3. Arabic/Hebrew Typography

Font considerations untuk Arabic:

```
/* Font Arabic umum */
[lang="ar"] {
  font-family: 'Noto Naskh Arabic', 'Amiri', 'Traditional Arabic', serif;
  line-height: 1.8; /* Arabic butuh line-height lebih besar */
  font-size: 1.05em; /* Arabic biasanya lebih kecil secara visual */
}

/* Fallback sans-serif untuk UI */
[lang="ar"] .ui-text {
  font-family: 'Noto Sans Arabic', 'Cairo', sans-serif;
}
```

Perbedaan tipografi:

Aspek	Latin	Arabic
Writing direction	LTR	RTL
Cursive	Tidak	Ya (connected)
Letter case	Upper/lower	No case
Numerals	0-9 (Western)	٩-٠ (Arabic-Indic)
Font size	Normal	Slightly larger needed
Line height	1.5	1.6-1.8
Font weight	Medium (400)	Regular (400) works fine

Arabic numerals formatting:

```
// Intl.NumberFormat untuk Arabic
const price = new Intl.NumberFormat('ar-SA', {
  style: 'currency',
```

```
currency: 'SAR',
}).format(1000);
// → "١٠٠٠٫٠٠" (Arabic-Indic digits)
```

4. API Translation — Accept-Language Header

Client request:

```
// fetch API dengan Accept-Language
const response = await fetch('/api/posts', {
  headers: {
    'Accept-Language': idn.language, // 'id' atau 'en'
  },
});
```

Express middleware:

```
// src/backend/middleware/i18n.ts
import { Request, Response, NextFunction } from 'express';

const SUPPORTED_LOCALES = ['en', 'id', 'ar'] as const;
type Locale = (typeof SUPPORTED_LOCALES)[number];
const DEFAULT_LOCALE: Locale = 'en';

// Parse Accept-Language: "id-ID,id;q=0.9,en;q=0.8"
function parseAcceptLanguage(header: string): Locale[] {
  return header
    .split(',')
    .map((entry) => {
      const [lang, qPart] = entry.split(';');
      const q = qPart ? parseFloat(qPart.split('=')[1]) : 1.0;
      return { lang: lang.trim().split('-')[0], q }; // ambil base language
    })
    .sort((a, b) => b.q - a.q) // sort by quality
    .map(({ lang }) => lang)
    .filter((l): l is Locale =>
      SUPPORTED_LOCALES.includes(l as Locale)
    );
}

export function i18nMiddleware(req: Request, _res: Response, next: NextFunction) {
  const acceptHeader = req.headers['accept-language'] || '';
  const locales = parseAcceptLanguage(acceptHeader);

  req.locale = locales[0] ?? DEFAULT_LOCALE;
  next();
}
```

Extend Express Request type:

```
// src/types/express.d.ts
declare namespace Express {
  interface Request {
    locale: 'en' | 'id' | 'ar';
  }
}
```

Route handler:

```
router.get('/posts', async (req, res) => {
  const locale = req.locale;
  const posts = await Post.find().populate('translations');

  // ambil translasi sesuai locale
  const result = posts.map((post) => ({
    id: post._id,
    title: post.translations[locale]?.title ?? post.translations['en']?.title,
    body: post.translations[locale]?.body ?? post.translations['en']?.body,
  }));

  res.json(result);
});
```

5. Database Schema untuk Multilingual Content

PostgreSQL — JSONB field:

```
CREATE TABLE posts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  slug VARCHAR(255) UNIQUE NOT NULL,
  translations JSONB NOT NULL DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- contoh data
INSERT INTO posts (slug, translations) VALUES (
  'hello-world',
  '{
    "en": {"title": "Hello World", "body": "This is English content."},
    "id": {"title": "Halo Dunia", "body": "Ini konten Bahasa Indonesia."},
    "ar": {"title": "مرحبا بالعالم", "body": "هذا المحتوى بالعربية."}
  }'
);
```

Query by locale:

```

SELECT
  slug,
  translations->'en'->'title' AS title_en,
  translations->'id'->'title' AS title_id,
  translations->'ar'->'title' AS title_ar
FROM posts;

```

Index untuk JSONB access:

```
CREATE INDEX idx_posts_translations ON posts USING GIN (translations);
```

Alternatif: Separate translation table (normalized)

```

CREATE TABLE posts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  slug VARCHAR(255) UNIQUE NOT NULL,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE post_translations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  post_id UUID REFERENCES posts(id) ON DELETE CASCADE,
  locale VARCHAR(10) NOT NULL CHECK (locale IN ('en', 'id', 'ar')),
  title TEXT NOT NULL,
  body TEXT NOT NULL,
  UNIQUE(post_id, locale)
);

```

Kapan pakai JSONB vs separate table:

JSONB	Separate Table
Sederhana, 1 tabel	Query per locale lebih cepat (index)
Update bulk translation	Translator bisa edit per bahasa
Schema flexible	Foreign key integrity
Cocok: < 5 bahasa, rare update	Cocok: banyak bahasa, frequent partial update

Mongoose (MongoDB) schema:

```

const postSchema = new Schema({
  slug: { type: String, required: true, unique: true },
  translations: {
    type: Map,
    of: {
      title: { type: String, required: true },
      body: { type: String, required: true },
    }
  }
});

```

```

    },
    default: {},
  },
}, { timestamps: true });

// method helper
postSchema.methods.translate = function (locale: string) {
  return this.translations.get(locale) ?? this.translations.get('en');
};

```

□ Kode Praktik

RTLWrapper Component

```

// src/components/RTLWrapper.tsx
import { useEffect } from 'react';
import { useTranslation } from 'react-i18next';

export function RTLWrapper({ children }: { children: React.ReactNode }) {
  const { i18n } = useTranslation();

  useEffect(() => {
    const dir = i18n.dir();
    const lang = i18n.language;

    document.documentElement.dir = dir;
    document.documentElement.lang = lang;

    // update CSS custom property untuk bg position dll
    document.documentElement.style.setProperty(
      '--dir-start',
      dir === 'rtl' ? 'right' : 'left'
    );
    document.documentElement.style.setProperty(
      '--dir-end',
      dir === 'rtl' ? 'left' : 'right'
    );
  }, [i18n.language]);

  return <>{children}</>;
}

```

Express Server Lengkap

```
npm install express cors
```

```
// server.ts
import express from 'express';
import cors from 'cors';
import { i18nMiddleware } from './middleware/i18n';

const app = express();
app.use(cors());
app.use(express.json());
app.use(i18nMiddleware);

app.get('/api/greeting', (req, res) => {
  const locale = req.locale;
  const greetings: Record<string, string> = {
    en: 'Hello!',
    id: 'Halo!',
    ar: '!!مرحبا!',
  };
  res.json({
    message: greetings[locale] ?? greetings['en'],
    locale,
  });
});

app.listen(3001, () => console.log('Server on :3001'));
```

□ AI Prompt Exercises

Latihan 1: CSS Logical Properties

Prompt untuk AI:

“Saya punya komponen Card React dengan avatar (kiri), judul, deskripsi, dan action button (kanan). Refactor CSS dari physical (left/right) ke logical properties (inline-start/inline-end). Sertakan layout flex dengan gap. Test: harus sama baik di LTR maupun RTL. Berikan CSS asli dan versi logical.”

Kriteria: Semua left/right hilang, ganti start/end. border, margin, padding, text-align semua logical.

Latihan 2: Express i18n Middleware + Testing

Prompt untuk AI:

“Tulis Express middleware i18n yang: (1) parse Accept-Language header, (2) set req.locale, (3) set response header Content-Language. Juga buat route GET /api/articles yang return data multilingual dari JSONB PostgreSQL. Sertakan test manual dengan curl.”

Kriteria: Handle q value fallback, Content-Language header, error jika locale tidak supported.

Latihan 3: Database Translasi Normalized

Prompt untuk AI:

“Bantu saya desain schema PostgreSQL untuk blog multilingual dengan separate translation table. Buat: posts, categories, post_translations, category_translations. Juga buat query untuk mendapatkan semua post dengan title di locale tertentu beserta category name-nya.”

Kriteria: Foreign key, unique constraint per locale, index pada locale, query efisien.

Latihan 4: RTL Image Mirroring

Prompt untuk AI:

“Buat komponen React RTLImage yang otomatis mirror image secara horizontal di RTL mode. Gunakan CSS transform: scaleX(-1). Tapi hanya untuk image yang punya arah (panah, gesture, directional icon). Kasih prop directional={true} untuk kontrol. Sertakan contoh: arrow-left.svg, hand-pointing.svg.”

Kriteria: Conditional mirror, animasi smooth untuk transisi RTL↔LTR, type safety.

□ Evaluasi

Indikator	Deskripsi
RTL layout CSS logical	html dir berubah, layout adjust otomatis No hardcoded left/right, pakai start/end

Indikator	Deskripsi
Accept-Language	Parse, fallback, Content-Language header
DB schema	JSONB atau separate table, query per locale
Arabic typography	Font, line-height, numeral handling

Sesi 04: Production i18n — Manajemen, Linting, SEO, Workflow

Durasi: 5 Jam

□ Tujuan Pembelajaran

Setelah sesi ini mahasiswa mampu:

- Memilih Translation Management System (TMS): lokal vs SaaS
- Menggunakan i18next-scanner untuk i18n linting
- Mengimplementasikan typesafe-i18n
- Melakukan lazy loading translations per locale (code splitting)
- Mengimplementasikan SEO hreflang tags
- Mendesain translation workflow (Dev → Translation → Review → Deploy)

□ Materi

1. Translation Management System (TMS)

Approach	Tool	Kelebihan	Kekurangan
Local	JSON files + git	Sederhana, version control, no cost	Manual sync, translator perlu akses repo
SaaS	Lokalise	UI terjemahan, auto-sync, peran (translator/reviewer)	Berbayar, dependency
SaaS	Crowdin	Community translation, integrasi Crowdin CLI	Resource untuk setup

Approach	Tool	Kelebihan	Kekurangan
SaaS	Phrase (ex PhraseApp) lengkap	Screenshot context, API	Pricing per seat

Rekomendasi untuk skala: - **Kecil (1-5 bahasa):** JSON + git + PR review workflow - **Medium (5-15 bahasa):** Lokalise atau Crowdin - **Enterprise (>15 bahasa):** Phrase + dedicated localization team

2. i18n Linting — i18next-scanner

`npm install --save-dev i18next-scanner`

Config:

```
// i18next-scanner.config.js
module.exports = {
  input: ['src/**/*.ts', 'src/**/*.tsx'],
  output: './',
  options: {
    debug: true,
    func: {
      list: ['t', 'i18n.t'], // fungsi yang memanggil t()
      extensions: ['.ts', '.tsx'],
    },
    trans: {
      component: 'Trans', // komponen Trans
      i18nKey: 'i18nKey',
      defaultsKey: 'defaults',
      extensions: ['.tsx'],
    },
    lngs: ['en', 'id', 'ar'],
    ns: ['common', 'home', 'auth'],
    defaultLng: 'en',
    resource: {
      loadPath: 'public/locales/{{lng}}/{{ns}}.json',
      savePath: 'public/locales/{{lng}}/{{ns}}.json',
    },
  },
};
```

Jalankan:

`npx i18next-scanner`

Apa yang dilakukan: - Scan semua file ts/tsx - Ekstrak key yang dipanggil `t('...')` - Generate file JSON dengan key baru - **TIDAK menghapus** key yang tidak terpakai (manual cleanup)

Tambahkan script di package.json:

```
{
  "scripts": {
    "il8n:scan": "il8next-scanner --config il8next-scanner.config.js",
    "il8n:missing": "il8next-scanner --config il8next-scanner.config.js --fail-on-missing"
  }
}
```

3. Typesafe i18n — typecheck translation keys

`npm install typesafe-il8next`

Type definition:

```
// src/il8n/il8n.d.ts
import 'il8next';
import type common from '../public/locales/en/common.json';
import type home from '../public/locales/en/home.json';
import type auth from '../public/locales/en/auth.json';

// Merge semua resources
interface Resources {
  common: typeof common;
  home: typeof home;
  auth: typeof auth;
}

// Extend il8next types
declare module 'il8next' {
  interface CustomTypeOptions {
    defaultNS: 'common';
    resources: Resources;
  }
}
```

Efek: TypeScript akan complain jika: - Key tidak ada di JSON → `t('nonexistent')` → error TS - Interpolation parameter salah → `t('greeting', { name: 'x' })` → error TS (harus name) - Count type mismatch

```
// ❌ OK
t('greeting', { name: 'Alice' });
```

```
// ❌ TypeScript error – properti 'nam' tidak ada di parameter greeting  
t('greeting', { nam: 'Alice' });
```

```
// ❌ TypeScript error – key 'nonexistent' tidak ada  
t('nonexistent');
```

4. Lazy Loading Translations — Code Splitting

Problem: Bundle semua locale JSON di entry point → bundle besar.

Solusi: Dynamic import per locale + React.lazy

```
// src/i18n/lazy-loader.ts  
import i18n from 'i18next';  
  
const localeLoaders: Record<string, () => Promise<Record<string, object>>> = {  
  en: () => import('../locales/en'),  
  id: () => import('../locales/id'),  
  ar: () => import('../locales/ar'),  
};  
  
export async function loadLocale(locale: string, ns: string = 'common') {  
  const loader = localeLoaders[locale];  
  if (!loader) throw new Error(`Locale ${locale} not supported`);  
  
  const resources = await loader();  
  const module = resources[ns];  
  
  if (module) {  
    i18n.addResourceBundle(locale, ns, module, true, true);  
  }  
}
```

Dengan React.lazy + Suspense:

```
// src/i18n/I18nProvider.tsx  
import { Suspense, useEffect, useState } from 'react';  
import i18n from './config';  
  
export function I18nSuspense({ children }: { children: React.ReactNode }) {  
  const [ready, setReady] = useState(i18n.isInitialized);  
  
  useEffect(() => {  
    if (!ready) {  
      i18n.on('initialized', () => setReady(true));  
    }  
  }, [ready]);  
}
```

```

    if (!ready) {
      return <div className="spinner">Loading translations...</div>;
    }

    return <Suspense fallback={<div className="spinner" />>{children}</Suspense>;
  }

```

Vite chunk splitting:

```

// vite.config.ts
export default defineConfig({
  build: {
    rollupOptions: {
      output: {
        manualChunks(id) {
          // separate locale files
          if (id.includes('/locales/')) {
            return 'locale-' + id.split('/locales/')[1].split('/')[0];
          }
        },
      },
    },
  },
});

```

5. SEO & hreflang Tags

hreflang — memberitahu Google versi bahasa halaman:

```

<link rel="alternate" hreflang="en" href="https://example.com/en" />
<link rel="alternate" hreflang="id" href="https://example.com/id" />
<link rel="alternate" hreflang="ar" href="https://example.com/ar" />
<link rel="alternate" hreflang="x-default" href="https://example.com/en" />

```

React component:

```

// src/components/HreflangTags.tsx
import { Helmet } from 'react-helmet-async';

const locales = ['en', 'id', 'ar'];

interface Props {
  currentPath: string; // e.g. /blog/hello-world
  siteUrl: string;      // e.g. https://example.com
}

export function HreflangTags({ currentPath, siteUrl }: Props) {

```

```

return (
  <Helmet>
    {locales.map((locale) => (
      <link
        key={locale}
        rel="alternate"
        hreflang={locale}
        href={` ${siteUrl}/${locale}${currentPath}`}
      />
    ))}
    <link
      rel="alternate"
      hreflang="x-default"
      href={` ${siteUrl}/en${currentPath}`}
    />
  </Helmet>
);
}

```

Next.js App Router — metadata:

```

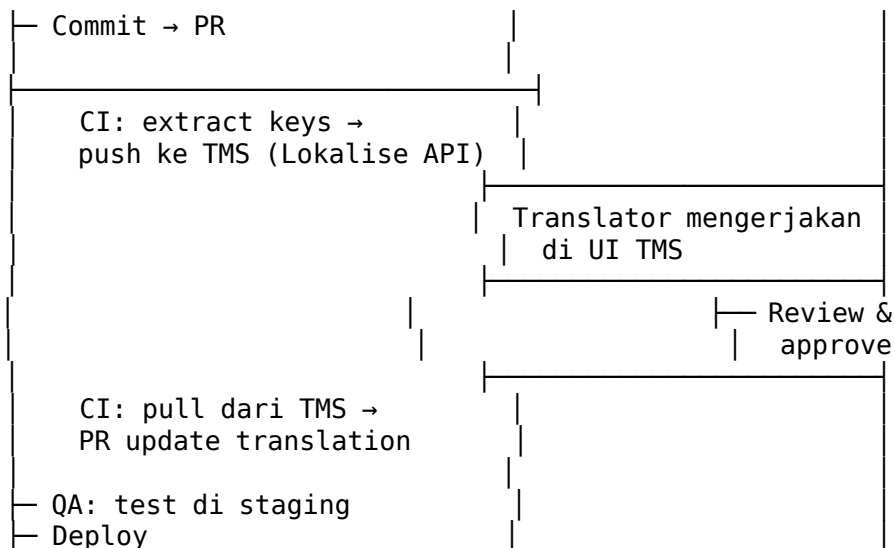
// app/[locale]/layout.tsx
export async function generateMetadata({ params: { locale } }: Props) {
  return {
    alternates: {
      languages: {
        'en': '/en',
        'id': '/id',
        'ar': '/ar',
        'x-default': '/en',
      },
    },
  };
}

```

SEO best practices i18n: - Gunakan subdirectory (/en, /id, /ar) — lebih disukai Google - **Jangan** gunakan query param (?lang=en) — crawling buruk - Satu canonical URL per locale - Hindari duplikat konten — hreflang + canonical - Sitemap terpisah per locale atau annotate satu sitemap

6. Translation Workflow

[Dev]	[Translator]	[Reviewer]
 — Tulis kode dengan key baru — Jalankan i18next-scanner	 	



Automation script:

```

#!/bin/bash
# scripts/sync-translations.sh
# Pull latest translations from Lokalise

LOKALISE_TOKEN=${LOKALISE_TOKEN}
PROJECT_ID=${PROJECT_ID}

echo "Pulling translations from Lokalise..."
lokalis2 file download \
  --token $LOKALISE_TOKEN \
  --project-id $PROJECT_ID \
  --format json \
  --unzip-to ./public/locales/

echo "Checking for missing keys..."
npx il8next-scanner --config il8next-scanner.config.js

echo "Committing changes..."
git add public/locales/
git commit -m "chore(il8n): sync translations from Lokalise"

```

CI/CD pipeline snippet (GitHub Actions):

```

# .github/workflows/il8n-sync.yml
name: il8n Sync
on:
  push:
    branches: [main]

```

```

    paths: ['src/**/*.ts', 'src/**/*.tsx']

jobs:
  extract:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci
      - run: npx il8next-scanner --config il8next-scanner.config.js
      - name: Push keys to Lokalise
        run: |
          lokalise2 file upload \
            --token ${ secrets.LOKALISE_TOKEN } \
            --project-id ${ secrets.LOKALISE_PROJECT_ID } \
            --file public/locales/en/common.json \
            --lang-iso en

```

□ Kode Praktik

Integration: Lazy Load + hreflang

```

// App.tsx – full flow
import { BrowserRouter, Route, Routes } from 'react-router-dom';
import { HelmetProvider } from 'react-helmet-async';
import { I18nSuspense } from './i18n/I18nProvider';
import { RTLWrapper } from './components/RTLWrapper';
import { LanguageSwitcher } from './components/LanguageSwitcher';
import { HreflangTags } from './components/HreflangTags';
import Home from './pages/Home';

function App() {
  return (
    <HelmetProvider>
      <BrowserRouter>
        <I18nSuspense>
          <RTLWrapper>
            <LanguageSwitcher />
            <HreflangTags
              currentPath="/"
              siteUrl="https://example.com"
            />
          </>
          <Routes>
            <Route path="/" element={<Home />} />
          </Routes>
        </I18nSuspense>
      </BrowserRouter>
    </HelmetProvider>
  );
}

```

```

        </RTLWrapper>
      </I18nSuspense>
    </BrowserRouter>
  </HelmetProvider>
);
}

```

Lokalise CLI Setup

```

# Install lokalise CLI
brew install lokalise/tap/lokalise2
# atau download dari https://github.com/lokalise/lokalise-cli-2-go

# Auth
export LOKALISE_TOKEN=your_api_token

# Download translations
lokalise2 file download \
  --project-id 123.abc \
  --format json \
  --unzip-to ./public/locales/

# Upload new keys
lokalise2 file upload \
  --file public/locales/en/common.json \
  --lang-iso en

```

□ AI Prompt Exercises

Latihan 1: i18next-scanner Setup

Prompt untuk AI:

“Bantu saya setup i18next-scanner untuk React TypeScript project. Ekstrak key dari fungsi t() dan komponen Trans. Namespace: common, home, auth, dashboard. Locale: en, id, ar. Output langsung ke public/locales/. Juga buat GitHub Actions workflow untuk auto-scan di setiap PR.”

Kriteria: Exclude test files, fail on update mode di CI, report missing keys sebagai PR comment.

Latihan 2: Typesafe Translation

Prompt untuk AI:

“Saya punya file translasi common.json dengan struktur: greeting ({{name}}), items ({{count}} plural), and nav (objek dengan home, about). Buat type declaration i18n.d.ts untuk membuat i18next type-safe. Sertakan contoh error TypeScript jika key salah atau parameter salah.”

Kriteria: Resource interface auto-generate dari JSON (atau manual), defaultNS common, interpolasi strict.

Latihan 3: Lazy Loading + Bundle Analyzer

Prompt untuk AI:

“Implementasi lazy loading translations dengan code splitting di Vite. Gunakan dynamic import tiap locale. Validasi bundle size dengan vite-bundle-analyzer. Target: masing-masing locale chunk < 5 KB gzipped. Sertakan visual report sebelum/ sesudah implementasi.”

Kriteria: Routes untuk en/id/ar, loading skeleton, manualChunks di vite.config.ts, analisis size.

Latihan 4: Translation Workflow Automation

Prompt untuk AI:

“Desain translation workflow lengkap untuk tim 2 developer + 1 translator + 1 reviewer. Tools: GitHub, Lokalise, i18next-scanner. Buat: (1) git branch strategy, (2) Lokalise project structure, (3) GitHub Actions workflow untuk auto sync, (4) PR template untuk translation update.”

Kriteria: Branch protection, auto-assign ke translator, review approval required, deploy hanya setelah review.

□ Evaluasi

Indikator	Deskripsi
i18next-scanner	Scan, extract, generate JSON otomatis
Type safety	TypeScript compile error untuk key invalid
Code splitting	Lazy loading per locale, bundle terpisah
SEO hreflang	Link alternate semua locale + x-default
Workflow	CI/CD pipeline, TMS sync, PR review
Bundle size	Masing-masing locale chunk termonitor

Indikator	Deskripsi
-----------	-----------